

memdb User Guide

Temporal graph + vector sidecar + key/value metadata, optimized as long-term memory for agents.

Version: Phase A (P16–P23)

Date: 2025-12-30

memdb User Guide

Filesystem-backed temporal graph + vector sidecar + key/value metadata, built to act as long-term memory for agents.

Version: Phase A (P16–P20) Date: 2025-12-29

1. What memdb is

memdb is a **temporal graph database** implemented on top of a **plain filesystem**:

- **Content layer (evidence)**: immutable “things you captured” (markdown, text, blobs, URIs). - **Entity layer (identity)**: stable IDs for people, repos, files, credentials, etc. - **Fact layer (edges)**: assertions with **validity intervals** (`validFrom`, `validTo`) and provenance (`sourceEventId`). - **State clock (canonical facts)**: a curated “what’s true now” view derived from observed facts. - **Vector sidecar**: optional embeddings you upsert yourself; used for semantic retrieval. - **KV metadata**: tags (key=value) on everything; indexed via `index / maintain`.

This gives you both clocks: - **Event clock**: “what happened (and why)” - **State clock**: “what is true right now”

2. Repo layout

```
apps/  
  memdb-web/          # placeholder UI package (future consumer)  
docs/  
  memdb-user-guide.md # this guide  
  mcp.md              # Claude/Codex MCP configs  
  auth.md             # API key + scopes + pack boundaries  
packages/  
  memdb-core/         # storage engine + indexes + maintenance  
  memdb-api/          # REST API server  
  memdb-cli/          # REST client CLI (mem ...)  
  memdb-mcp/          # MCP stdio server (tools for agents)  
packs/  
  *.pack.json         # pack definitions (core/coding_assistant/identity_verifier)
```

3. Storage layout on disk (MEMDB_ROOT)

By default memdb stores data under:

- `MEMDB_ROOT=./memdb`

Key subfolders:

```
memdb/  
  packs/                # pack copies used at runtime  
  entities/<type>/<id>.json  
  events/<YYYY>/<MM>/<DD>/<id>.json  
  edges/<YYYY>/<MM>/<DD>/<id>.json  
  content/<YYYY>/<MM>/<DD>/<id>.json  
  blobs/<YYYY>/<MM>/<DD>/<id>.bin  
  kv/  
    pointers.json        # "latest known location" for each entity/event/edge  
    adj_current/         # adjacency lists (current pointers)  
    timeline/            # per-entity event timelines  
    tags/                # inverted indexes for tags (built by index)  
    vectors/             # vector sidecar items  
    deltas/              # replay log segments (rebuilt by index)  
    manifests/           # per-entity manifests (maintenance)  
    auth/keys.json       # optional API keys (P18)
```

You can back up memdb by copying this directory (it is append-heavy; see the Ops section for best practices).

4. Running memdb

4.1 Prerequisites

- Deno (recent stable) - A writable filesystem

4.2 Start the API server

From repo root:

```
export MEMDB_ROOT=./memdb  
deno task api  
# listens on http://localhost:8787
```

Health check:

```
curl -sS http://localhost:8787/healthz
```

4.3 (Optional) Start MCP server (agent tool surface)

In another terminal:

```
export MEMDB_API_URL=http://localhost:8787  
export MEMDB_API_KEY=memdb_dev_key # if auth enabled  
deno task mcp
```

See docs/mcp.md for Claude Code and OpenAI Codex configs.

4.4 Use the CLI (REST client)

```
export MEMDB_API_URL=http://localhost:8787
deno task mem init
deno task mem index
```

New in P23: - mem search-hybrid uses /v1/search/hybrid - mem search-around uses /v1/search/around (subgraph retrieval)

Examples:

```
mem search-hybrid --kind facts --pack coding_assistant --q "timeout" --status canonical
mem search-around --kind facts --pack coding_assistant --root ent_repo_... --depth 2 --q "timeout" --status canonical
```

5. Packs and schema governance

A **pack** is a schema boundary: - allowed entityTypes - allowed predicates - allowed eventKinds - required tags / retention defaults / embedding restrictions

Pack definitions live in:

- packs/<name>.pack.json

Example templates included: - core - coding_assistant - identity_verifier

5.1 Creating a new pack

1) Copy an existing pack:

```
cp packs/coding_assistant.pack.json packs/my_pack.pack.json
```

2) Edit: - name - allowed entity types / predicates / event kinds - policy requirements

3) Restart API or run mem init to copy packs into MEMDB_ROOT/packs.

Best practice: treat packs like “published language” per agent type. Keep them small and explicit.

6. Data model and ID rules

6.1 Entities (identity layer)

Entity fields (conceptually):

Field	Meaning
`id`	`ent_...`
`type`	pack-defined type, e.g. `Repo`, `File`, `Person`
`key`	stable external key (your choice)
`tags`	metadata (`pack`, `retention`, etc.)

Create entity:

```
curl -sS -X POST http://localhost:8787/v1/entities -H "content-type: application/json" -d '{
  "pack": "coding_assistant",
  "type": "Repo",
  "key": "memdb",
  "tags": { "owner": "srdjan" }
}'
```

6.2 Content (evidence layer)

Content is immutable evidence: text, a URI, or a blob (base64).

Create content (text):

```
curl -sS -X POST http://localhost:8787/v1/content -H "content-type: application/json" -d '{
  "pack": "coding_assistant",
  "mime": "text/markdown",
  "text": "We increased timeout to 30s because cold starts were failing.",
  "tags": { "source": "design-review" }
}'
```

List content by pack:

```
curl -sS "http://localhost:8787/v1/content?pack=coding_assistant"
```

6.3 Events (event clock)

Events are the audit trail. They link agents to evidence and entities.

Create an event:

```
curl -sS -X POST http://localhost:8787/v1/events -H "content-type: application/json" -d '{
  "pack": "coding_assistant",
  "kind": "decision_recorded",
  "agentId": "claude-code",
  "refs": [
```

```
{ "kind": "entity", "id": "ent_..." },
{ "kind": "content", "id": "cnt_..." }
],
"tags": { "topic": "timeouts" }
}'
```

6.4 Edges / Facts (temporal assertions)

An edge is a fact with: - predicate (pack-defined) - subject s (entity ID) - object o (entity ID) - validFrom / validTo (temporal validity) - sourceEventId (provenance) - tags (including status for canonical lane)

Add an observed fact:

```
curl -sS -X POST http://localhost:8787/v1/edges -H "content-type: application/json" -d '{
  "pack": "coding_assistant",
  "predicate": "config",
  "s": "ent_file_server_ts",
  "o": "ent_timeout_30000",
  "sourceEventId": "evt_...",
  "validFrom": "2025-12-29T00:00:00.000Z",
  "confidence": 0.7,
  "tags": { "status": "observed" }
}'
```

Retract a fact (close its interval):

```
curl -sS -X POST http://localhost:8787/v1/edges/retract -H "content-type: application/json" -d '{
  "pack": "coding_assistant",
  "predicate": "config",
  "s": "ent_file_server_ts",
  "o": "ent_timeout_5000",
  "sourceEventId": "evt_...",
  "validTo": "2025-12-29T00:00:00.000Z",
  "confidence": 1.0,
  "tags": { "reason": "superseded" }
}'
```

7. Querying (event clock)

7.1 Neighbors (temporal adjacency)

```
curl -sS "http://localhost:8787/v1/neighbors?entityId=ent_...&asOf=2025-12-29T00:00:00.000Z&pack=coding_as"
```

This returns edges active at asOf (observed + canonical + synthesized unless you filter via tags yourself).

7.2 Path queries

```
curl -sS "http://localhost:8787/v1/path?from=ent_...&to=ent_...&asOf=2025-12-29T00:00:00.000Z&pack=coding_...
```

7.3 Timeline (per-entity event history)

```
curl -sS "http://localhost:8787/v1/timeline?entityId=ent_..."
```

7.4 Pattern queries (canonical-first)

When you need more than neighbors/path but don't want a full query language, use the JSON pattern query.

Endpoint

- POST /v1/query/pattern

The query is **anchored**: each clause must have at least one side already bound (a concrete s/o, or a variable bound by a previous clause). This keeps it deterministic and avoids full scans.

Example: find config facts for all files contained by a repo (canonical lane)

```
curl -sS -X POST http://localhost:8787/v1/query/pattern \
-H "content-type: application/json" \
-d '{
  "pack": "coding_assistant",
  "asOf": "2025-12-29T00:00:00.000Z",
  "status": "canonical",
  "clauses": [
    { "predicate": "contains", "s": "ent_repo_...", "oVar": "file", "oType": "File" },
    { "predicate": "config", "sVar": "file", "oVar": "cfg" }
  ],
  "return": ["file", "cfg"],
  "limit": 50,
  "explain": true
}'
```

Outputs

- results[].bindings are the variable bindings (file, cfg) - results[].matches are the matched edges (useful for provenance) - with explain=true, you also get per-clause cardinalities

8. Canonical state (state clock) and resolution (P16)

Agents will produce contradictory “observed” facts. The fix is to treat truth as a derived product.

8.1 Preview resolution

POST /v1/resolve

- analyzes a neighborhood and proposes merges / supersessions - ****does not write canonical facts**** unless persist=true (preview by default)

```
curl -sS -X POST http://localhost:8787/v1/resolve -H "content-type: application/json" -d '{
  "entityId": "ent_...",
  "pack": "coding_assistant",
  "asOf": "2025-12-29T00:00:00.000Z",
  "persist": false
}'
```

8.2 Apply resolution (the “obvious move”)

POST /v1/resolve/apply

- emits ****canonical**** facts (tags.status="canonical") - optionally emits ****synthesized**** facts (tags.status="synthesized", predicate prefixed with synth/...) - emits a sys/fact_resolution_applied event (pack core) as provenance

```
curl -sS -X POST http://localhost:8787/v1/resolve/apply -H "content-type: application/json" -d '{
  "entityId": "ent_...",
  "pack": "coding_assistant",
  "asOf": "2025-12-29T00:00:00.000Z",
  "emitSynthesized": true,
  "agentId": "memdb-resolver"
}'
```

8.3 Read canonical-only state (the state clock endpoint)

GET /v1/state/neighbors

```
curl -sS "http://localhost:8787/v1/state/neighbors?entityId=ent_...&asOf=2025-12-29T00:00:00.000Z&pack=coding_assistant"
```

****This endpoint is what other agents should rely on**** when they need “truth right now”.

8.4 Automatic interval closing for conflicts (P20)

Sometimes you want “latest wins” semantics for *observed* facts (without waiting for a resolver run). memdb supports an ****optional pack policy**** that will ****auto-close**** conflicting intervals when a new observed fact arrives.

In your pack file:

```
{
  "policy": {
```



```

    "conflictPolicies": [
      { "predicate": "repo:defaultBranch", "uniqueness": "one_per_subject", "lane": "observed" }
    ]
  }
}

```

Behavior:

- On POST /v1/edges for that predicate, memdb finds currently-active edges with the same (predicate, subject) and a different object. - It emits a sys/auto_retract event (pack core) and writes retraction edges whose validTo = new.validFrom. - Canonical / synthesized edges are *not* auto-closed (those are handled by resolution).

This is deliberately conservative: it keeps the “event clock” rich, while reducing obvious contradictions.

8.5 Scheduling resolution (queue + worker) (P20)

To make the “state clock” stay fresh for multiple agents, memdb provides a **resolution queue**:

- POST /v1/resolve/enqueue (write scope): add a job - GET /v1/resolve/queue (ops scope): inspect jobs - a lightweight worker inside memdb-api drains jobs and runs resolve/apply

Enqueue:

```

curl -sS -X POST http://localhost:8787/v1/resolve/enqueue \
  -H "content-type: application/json" \
  -H "x-memdb-api-key: $MEMDB_API_KEY" \
  -d '{
    "pack": "coding_assistant",
    "entityId": "ent_...",
    "priority": 10,
    "reason": "after tool-run"
  }'

```

Worker controls (env vars):

- MEMDB_RESOLVE_WORKER=false disables the worker - MEMDB_RESOLVE_WORKER_MS=2000 sets the polling interval (default 2000ms)

8.6 State clock diffs and snapshots (P20)

When debugging agent behavior, you often want *changes*, not just a point-in-time view.

Diff (two timestamps):

```
GET /v1/state/diff?entityId=...&t1=...&t2=...&pack=...
```

Returns { added, removed, unchanged } edges (canonical lane).

****Subgraph snapshot (BFS):****

```
GET /v1/state/subgraph?rootEntityId=...&asOf=...&pack=...&depth=2
&maxEdges=500
```

Returns { nodes , edges } rooted at the entity, walking canonical edges outward.

These endpoints are intentionally “small but useful”: they cover 80% of debugging workflows without forcing a full query language.

9. Search: text + vectors + tags

9.1 Simple text search

```
GET /v1/search?kind=entities|facts|content&q=...&pack=...
```

```
curl -sS "http://localhost:8787/v1/search?kind=facts&q=status:canonical&pack=coding_assistant"
```

This is substring matching over: - ids, keys, predicates - edgeKey - tag strings (k:v)

9.2 Vector sidecar (*caller-provided embeddings*)

memdb does ****not**** generate embeddings for you. You upsert them.

Upsert an embedding:

```
curl -sS -X POST http://localhost:8787/v1/vectors/upsert -H "content-type: application/json" -d '{
  "id": "ent_...",
  "kind": "entity",
  "pack": "coding_assistant",
  "embedding": [0.01, 0.02, 0.03],
  "tags": { "model": "text-embedding-3-large" }
}'
```

Search embeddings:

```
curl -sS -X POST http://localhost:8787/v1/vectors/search -H "content-type: application/json" -d '{
  "query": [0.01, 0.02, 0.03],
  "topK": 10,
  "filter": { "pack": "coding_assistant", "kind": "entity" }
}'
```

9.3 Tag indexing

Tags are indexed by index and partially refreshed by maintain.

- POST /v1/index rebuilds global indexes deterministically - POST /v1/maintain refreshes per-entity materializations

9.3 Hybrid search (vector + text + recency)

This endpoint lets you combine: - **vector similarity** (if you provide vector) - **text match** (if you provide q) - **recency boost** (based on record timestamps: createdAt / capturedAt / recordedAt)

POST /v1/search/hybrid

Request body:

```
{
  "kind": "entities | content | facts",
  "pack": "optional pack name",
  "status": "facts only: canonical | observed | synthesized",
  "asOf": "ISO time for recency scoring (default: now)",
  "q": "optional text query",
  "vector": [0.12, -0.03, ...],
  "filterTags": { "k": "v" },
  "limit": 10,

  "alpha": 0.7,
  "beta": 0.2,
  "gamma": 0.1,
  "halfLifeDays": 30
}
```

Notes: - You can send **only** q, **only** vector, or both. - filterTags are **exact-match AND filters** applied to returned records. - status filters only apply to facts (edges). - Cosine similarity is mapped from [-1..1] → [0..1] before weighting.

Response:

```
{
  "items": [
    {
      "id": "...",
      "score": 0.83,
      "scoreBreakdown": { "vector": 0.91, "text": 0.50, "recency": 0.74 },
      "record": { "...": "..." }
    }
  ]
}
```

Example: semantic + recency for recent canonical facts about a repo:

```
curl -sS -X POST http://localhost:8787/v1/search/hybrid \
-H "content-type: application/json" \
-d '{
  "kind": "facts",
  "pack": "coding_assistant",
  "status": "canonical",
  "vector": [0.01, 0.02, 0.03],
}
```

```

    "limit": 10,
    "gamma": 0.2,
    "halfLifeDays": 14
  }'

```

Example: hybrid query + tag filter:

```

curl -sS -X POST http://localhost:8787/v1/search/hybrid \
-H "content-type: application/json" \
-d '{
  "kind": "content",
  "pack": "identity_verifier",
  "q": "insurance stopped covering",
  "filterTags": { "source": "meeting_transcript" },
  "limit": 20
}'

```

****MCP note:**** the MCP tool `memdb.search` uses `/v1/search/hybrid` by default, so Claude Code / Codex get hybrid scoring without special handling.

9.4 Subgraph hybrid search (search around) (P23)

When agents already have an anchor entity (repo, subject, session, case, etc.), global search is often noisy. ****Subgraph hybrid search**** does:

- 1) Expand a ****small canonical neighborhood**** around the anchor (BFS by depth)
- 2) Score only items inside that neighborhood using ****vector + text + recency****
- 3) Return the top results with a score breakdown

This gives you **high-signal, pack-scoped** context that fits agent windows.

REST

POST `/v1/search/around`

Example: canonical facts near a repo

```

curl -sS -X POST http://localhost:8787/v1/search/around \
-H "content-type: application/json" \
-H "x-api-key: $MEMDB_API_KEY" \
-d '{
  "kind": "facts",
  "rootEntityId": "ent_repo_...",
  "pack": "coding_assistant",
  "depth": 2,
  "status": "canonical",
  "q": "timeout cold start",
  "halfLifeDays": 14,
  "limit": 10
}'

```

Example: content near an identity verification subject

```
curl -sS -X POST http://localhost:8787/v1/search/around \
-H "content-type: application/json" \
-H "x-api-key: $MEMDB_API_KEY" \
-d '{
  "kind": "content",
  "rootEntityId": "ent_subject_...",
  "pack": "identity_verifier",
  "depth": 2,
  "q": "insurance stopped covering",
  "limit": 10
}'
```

Safety caps: - maxNodes (default 1000) - maxEdges (default 5000)

These prevent pathological neighborhoods from overwhelming the process.

MCP

Tool: `memdb.search.around`

This is the recommended call for Claude Code / Codex when you have the anchor entity id.

```
{
  "kind": "facts",
  "rootEntityId": "ent_...",
  "pack": "coding_assistant",
  "depth": 2,
  "status": "canonical",
  "q": "timeout",
  "limit": 10
}
```

Recommended agent flow: - write evidence (content) + trace event - write observed edges - enqueue resolution (`memdb.resolve.enqueue`) or apply directly - retrieve via `memdb.search.around`

10. Maintenance and operations

10.1 Initialize and rebuild indexes

```
deno task mem init
deno task mem index
# or:
curl -sS -X POST http://localhost:8787/v1/index -H "content-type: application/json" -d '{}'
```

10.2 Health checks (per entity)

```
curl -sS "http://localhost:8787/v1/health?entityId=ent_...&asOf=2025-12-29T00:00:00.000Z&pack=coding_assi"
```

10.3 Maintain (per entity)

```
curl -sS -X POST http://localhost:8787/v1/maintain -H "content-type: application/json" -d '{"entityId": "..."}
```

10.4 Maintain-all (batch)

```
curl -sS -X POST http://localhost:8787/v1/maintain-all -H "content-type: application/json" -d '{"pack": "..."}
```

10.5 Report (stats)

```
curl -sS -X POST http://localhost:8787/v1/report -H "content-type: application/json" -d '{"pack": "..."}
```

10.x Metrics (Prometheus text format) (P20)

GET /metrics (requires ****ops**** scope)

```
curl -sS http://localhost:8787/metrics \
-H "x-memdb-api-key: $MEMDB_API_KEY"
```

Emits a minimal Prometheus exposition with:

- memdb_requests_total{method,path} request counter - memdb_errors_total{path} error counter - memdb_resolution_queue_length gauge - resolution worker counters (jobs started/succeeded/failed)

This is intentionally lightweight; it's enough to plug into a dashboard and start watching: queue length, resolution stability, and API load.

10.x Correlation IDs (P20)

If you send `x-correlation-id`, memdb echoes it back on responses. If you don't, memdb generates one per request.

This is a small addition that becomes extremely valuable once multiple agents and multiple tools start writing into the same pack.

11. Agent-friendly helpers (P19)

11.1 Decision trace helper: one call = content + event + observed facts

POST /v1/trace/decision

```
curl -sS -X POST http://localhost:8787/v1/trace/decision -H "content-type: application/json" -d '{
  "pack": "coding_assistant",
  "agentId": "claude-code",
  "kind": "decision_recorded",
  "content": { "mime": "text/markdown", "text": "Increase timeout to 30s for cold starts." },
  "claims": [
    { "predicate": "config", "s": { "type": "File", "key": "src/server.ts" }, "o": { "type": "ConfigKey"
  ]
}'
```

11.2 Bootstrap helper

POST /v1/bootstrap

```
curl -sS -X POST http://localhost:8787/v1/bootstrap -H "content-type: application/json" -d '{
  "pack": "coding_assistant",
  "template": "coding_assistant",
  "repoKey": "my-repo",
  "branchKey": "main",
  "files": ["src/main.ts", "src/server.ts"]
}'
```

12. Auth, scopes, and pack boundaries (P18)

memdb supports API keys with: - scopes: read, write, ops - pack allowlists - rate limiting (token bucket)

12.1 Configure keys

Create:

MEMDB_ROOT/kv/auth/keys.json

```
{
  "keys": [
    { "id": "dev", "key": "memdb_dev_key", "scopes": ["read","write","ops"], "packs": ["*"], "rate": { "r
    { "id": "ro_ca", "key": "memdb_ro_ca", "scopes": ["read"], "packs": ["coding_assistant"], "rate": { "
  ]
}
```

12.2 Enforce auth (even in dev)

```
export MEMDB_REQUIRE_AUTH=true
deno task api
```

12.3 Send the key

Either:

- Authorization: Bearer <key> - x-api-key: <key>

****Pack boundaries are enforced**** on endpoints that accept pack parameters.

13. MCP (Claude Code, Codex) usage patterns (P17)

The MCP server is the stable “tool surface” for agent runtimes.

Recommended pattern:

1) Agents write ****observed**** facts (with evidence refs). 2) After bursts, call `memdb.resolve.apply` for affected entities. 3) Agents read via `memdb.state.neighbors` (canonical-only).

See `docs/mcp.md` for the exact config snippets.

14. Two concrete templates: coding assistant vs identity verifier

14.1 Coding assistant memory (recommended spine)

Entities: - Repo, Branch, File, Symbol, Issue, Decision, ConfigKey

Facts: - contains, references, implements, config, decided, dependsOn

Typical flow: 1) trace/decision on changes or review notes 2) link to files/symbols 3) resolve/apply 4) state reads for “what is current config / current design decision”

14.2 Identity verifier memory (recommended spine)

Entities: - Subject, Credential, Issuer, Verifier, Evidence, Policy, Session

Facts: - issued, presented, verified, revoked, requires, satisfies, assessedRisk

Typical flow: 1) content = evidence record (transcript / document / proof) 2) event = verification run 3) edges = claims + policy outcomes (observed) 4) resolve/apply yields canonical current status for a subject/session

15. Troubleshooting

“Pack validation failed”

- You tried to write an entity type, predicate, or event kind not in the pack’s allowlist. - Fix the pack definition in `packs/*.pack.json` or change the write.

“Forbidden tag for embeddings”

- Pack policy can disallow certain tags from being indexed in vectors.

“Missing sourceEventId”

- All edges must point to an event. Create an event first (or use `trace/decision`).

Indexes look stale

- Run `POST /v1/maintain` for the entity. - If in doubt, run `POST /v1/index` (global rebuild).

Appendix A: REST endpoint map

Endpoint	Method	Purpose
<code>/healthz`</code>	GET	server health
<code>/v1/init`</code>	POST	initialize packs + folders
<code>/v1/index`</code>	POST	rebuild pointers/adjacency/timeline/tag indexes
<code>/v1/entities`</code>	POST/GET	create/list entities
<code>/v1/entities/:id`</code>	GET	get entity
<code>/v1/entity-ids`</code>	GET	list all entity IDs
<code>/v1/content`</code>	POST/GET	create/list content
<code>/v1/content/:id`</code>	GET	get content record
<code>/v1/events`</code>	POST	create event
<code>/v1/events/:id`</code>	GET	get event
<code>/v1/edges`</code>	POST	assert fact
<code>/v1/edges/retract`</code>	POST	retract fact
<code>/v1/neighbors`</code>	GET	event-clock neighbor query
<code>/v1/state/neighbors`</code>	GET	state-clock canonical neighbor query
<code>/v1/path`</code>	GET	path query
<code>/v1/timeline`</code>	GET	per-entity timeline
<code>/v1/search`</code>	GET	simple text search
<code>/v1/vectors/upsert`</code>	POST	upsert embedding

`/v1/vectors/search`	POST	similarity search
`/v1/resolve`	POST	resolver preview
`/v1/resolve/apply`	POST	emit canonical/synth facts
`/v1/trace/decision`	POST	one-call decision trace
`/v1/bootstrap`	POST	template seeding
`/v1/health`	GET	per-entity health
`/v1/maintain`	POST	per-entity maintenance
`/v1/maintain-all`	POST	batch maintenance
`/v1/report`	POST	stats report